

ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 1) BASIC SCENE

COURSE/SUBJECT: APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

COURSE NO.: ITE-18

INSTRUCTOR/PROF: JAMES EARL D. CUBILLAS, M.Sc., ECT

Objective:

This laboratory manual is designed to guide you through creating a basic 3D scene using the Three.js library. By the end of this lab, you will have built a simple scene containing a 3D object, a camera, and a renderer, and you will have learned how to display this scene on a webpage.

Introduction

For our first lesson, we will make Three.js work the most straightforward way: no bundler, no dependency, no modules, just an HTML file and some JavaScript.

Base files

To begin, create a plain index.html file:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Basic Scene</title>
</head>
<body>
  <script src="./script.js"></script>
</body>
</html>
```

And a simple script.js file:

```
console.log('Hello Three.js')
```

Open the **index.html** in your browser. To do that, you can try to double-click on the file. If it doesn't work or opens with the wrong browser, you can drag and drop the file onto the right browser. Even if Three.js will work on most browsers, I recommend using a developer-friendly browser like **Chrome** or **Firefox**.

Now open the **Developer Tools**.

You can right-click anywhere on the page and choose Inspect or press **F12** on Windows and **CMD + OPTION + I** on MacOS.

Then navigate to the **Console** tab at the top of the **Developer Tools**.

You should always keep the Console open to see potential errors and warnings.

How to load Three.js

Now we need to load the Three.js library. There are many ways of doing it. We will simply download the library and load it using `<script>`.

Go to <https://threejs.org/> and click on the download button to download a zip file and unzip it. The file is quite heavy, but don't worry, we only need one file from it.

Go to the build/ folder and copy the three.min.js file to your project.

We can now load the Three.js library at the end of the `<body>`, right before closing it:

```
<script src="./three.min.js"></script>
<script src="./script.js"></script>
```

Make sure to load three.min.js before your script.js; otherwise, your script will not know what's inside the three.min.js file.

How to use Three.js

Inside our script.js file, we now have access to a variable named THREE. Be careful and always write it using uppercase.

If you `console.log()` this variable, you'll see that there is a lot going on inside:

```
console.log(THREE)
```

The **THREE** variable contains most classes and properties you might need on a classic Three.js project. Unfortunately, not all classes are included in this variable, but we will see later how to access them.

To use one of those classes, you need to instantiate it. For example, if you want to create a scene, write **const scene = new THREE.Scene()**. If you want to create a sphere geometry, write **const sphereGeometry = new THREE.SphereGeometry(1.5, 32, 32)** —We'll investigate these later.

First scene

It's time to create our scene and produce something on the screen.

We need 4 elements to get started:

1. A scene that will contain objects
2. Some objects
3. A camera
4. A renderer

Scene

The scene is like a container. You place your objects, models, particles, lights, etc., in it, and at some point, you ask Three.js to render that scene.

To create a scene, use the [Scene](#) class:

```
// Scene
const scene = new THREE.Scene()
```

Objects

Objects can be many things. You can have primitive geometries, imported models, particles, lights, etc.

We will start with a simple red cube.

To create that red cube, we need to create a type of object named [Mesh](#). A [Mesh](#) combines a geometry (the shape) and a material (how it looks).

There are many geometries and materials, but we will keep things simple for now and create a [BoxGeometry](#) and a [MeshBasicMaterial](#).

To create the geometry, we use the [BoxGeometry](#) class with the first 3 parameters corresponding to the box's size.

```
// Object
const geometry = new THREE.BoxGeometry(1, 1, 1)
```

We use the **MeshBasicMaterial** class with one parameter: an object {} containing all the options to create the material. All we need is to specify its **color** property.

There are many ways to specify a color in Three.js. You can send it as a JS hexadecimal **0xff0000**, you can send it as a string hexadecimal **'#ff0000'**, you can use color names like **'red'**, or you can send an instance of the [Color](#) class —we'll cover more about it later.

```
// Object
const geometry = new THREE.BoxGeometry(1, 1, 1)
const material = new THREE.MeshBasicMaterial({ color: 0xff0000 })
```

We use the Mesh class to create the final mesh and send the **geometry** and the **material** as parameters.

```
// Object
const geometry = new THREE.BoxGeometry(1, 1, 1)
const material = new THREE.MeshBasicMaterial({ color: 0xff0000 })
const mesh = new THREE.Mesh(geometry, material)
```

You can now add your mesh to the scene by using the **add(...)** method:

```
scene.add(mesh)
```

If you don't add an object to the scene, you won't be able to see it.

Camera

The camera is not visible. It's more like a theoretical point of view. When we render your scene, it will be from that camera's point of view.

You can have multiple cameras, just like on a movie set, and you can switch between those cameras as you please. Usually, we only use one camera.

There are different types of cameras, and we will talk about these in a future lesson. For now, we simply need a camera that handles perspective (making close objects look more prominent than far objects).

To create the camera, we use the [PerspectiveCamera](#) class.

There are two essential parameters we need to provide.

The field of view

The field of view is how large your vision angle is. Using a very large angle, you can see in every direction simultaneously but with much distortion, because the result will be drawn on a small rectangle. If you use a small angle, things will look zoomed in. The field of view (or **fov**) is expressed in degrees and corresponds to the vertical vision angle. For this exercise, we will use a **75** degrees angle.

The aspect ratio

In most cases, the aspect ratio is the width of the canvas divided by its height. We haven't specified any width or height for now, but we will need to later. In the meantime, we will create an object with temporary values to reuse.

Don't forget to add your camera to the scene. Everything should work without adding the camera to the scene, but it might result in bugs later:

```
// Sizes
const sizes = {
  width: 800,
  height: 600
}

// Camera
const camera = new THREE.PerspectiveCamera(75, sizes.width / sizes.height)
scene.add(camera)
```

Renderer

The renderer's job is to do the render. I bet you didn't see that coming.

We will simply ask the renderer to render our scene from the camera's point of view, and the result will be drawn into a canvas. You can create the canvas by yourself, or let the renderer generate it and then add it to your page. We will add the canvas to the HTML for this exercise and send it to the renderer.

Create the **<canvas>** element **before** you load the scripts and give it a class:

```
<canvas class="webgl"></canvas>
```

We use the WebGLRenderer class with one parameter: an object {} containing all the options to create the renderer. We must specify the **canvas** property corresponding to the **<canvas>** we added to the page.

Create a **canvas** variable at the start of the code, then fetch and store in it the element we created in the HTML using **document.querySelector(...)**.

Assigning the canvas to a variable is better because we'll use it for other purposes in the next lessons.

We also need to update the size of your renderer with the **setSize(...)** method using the **sizes** object we created earlier. The **setSize(...)** method will automatically resize our **<canvas>** accordingly:

```
// Canvas
const canvas = document.querySelector('canvas.webgl')

// ...

// Renderer
const renderer = new THREE.WebGLRenderer({
  canvas: canvas
})
renderer.setSize(sizes.width, sizes.height)
```

You won't be able to see anything right now, but your canvas is there and has been resized accordingly. You can use the Developer Tools to inspect the **<canvas>** if you are curious.

First render

It's time to work on our first render. Call the **render(...)** method on the renderer and send it the **scene** and the **camera** as parameters:

Still nothing? Here's the issue: we haven't specified our object's position, nor our camera's. Both are in the default position, which is the center of the scene, and we can't see an object from its inside (by default).

We need to move things.

We can access multiple properties on each object, such as **position**, **rotation**, and **scale**. For now, use the **position** property to move the camera backward.

The **position** property is an object with three relevant properties: **x**, **y**, and **z**. By default, Three.js considers the forward/backward axis to be **z**.

To move the camera backward, we need to provide a positive value to that property. You can do that anywhere once you've created the **camera** variable, yet it has to happen before you do the render:

```
const camera = new THREE.PerspectiveCamera(75, sizes.width / sizes.height)
camera.position.z = 3
scene.add(camera)
```

ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 2) LOCAL SERVER

COURSE/SUBJECT: APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

COURSE NO.: ITE-18

INSTRUCTOR/PROF: JAMES EARL D. CUBILLAS, M.Sc., ECT

Introduction

The way we loaded Three.js in the previous lesson was the easiest. Unfortunately, it has two main limitations.

First, we only have access to “core” classes. We can do a lot with dozens of classes in this core, but we don’t master everything. For example, in a future lesson, we will need access to the **OrbitControls** class, which is unavailable in the core classes.

Secondly, your browser won’t let JavaScript execute instructions when opening an HTML file like this. For example, you won’t be able to load local files like textures or models. This is a good thing for security reasons. You don’t want a script to be able to load files from your computer just because you opened an HTML file that you thought was safe.

But still, we need to be able to run JavaScript code as if it were an online website, and to do that, we need to run a local server.

There are many ways to handle those issues, but a “build tool” or “bundler” is the simplest solution.

The state of build tools

Many build tools are available these days, and you’ve probably heard of some of them, like Webpack, Vite, Gulp, Parcel, etc.

They have various features with pros and cons, but we will use a very specific one in the following lessons.

Nowadays, the most popular build tool is Webpack. It’s widely used, it has a great community, and you can do a lot with it. But while Webpack is the most popular, it’s not the most appreciated.

The most appreciated build tool these days is [Vite](#) (the French word for “quick,” pronounced /vit/, like “veet”). It’s faster to install and run and less prone to bugs. Ultimately, the developer experience is much better.

Vite

As mentioned earlier, [Vite](#) is a build tool. We will write web code like HTML/CSS/JS, and Vite will build the final website. It’ll also do many things like optimizations, cache breaking, source mapping, running a local server, etc.

While Vite handles the most basic needs, we can also add plugins to handle more features like exotic languages, or special files. We are going to add a plugin later in the course, which will be able to handle GLSL files to create custom shaders.

Vite was created by Evan You, the creator of Vue.js, and is highly maintained by hundreds of developers.

Node.js

First, you need to have [Node.js](#) installed on your computer.

Node.js enables running JavaScript on your computer outside of a browser. It's great to run various tools like Vite, it has been around for many years and is very popular.

If you don't know if Node.js is already installed or which version is installed, open your Terminal (MacOS) or Command Prompt (Windows) and run **node -v**.

If the answer tells you that the **node** command isn't recognized, it's not installed.

If it's installed, the answer will contain the current version. Make sure it's updated to the last version. Vite currently works with version **14.18** and above, but I recommend always installing the LTS (Long Term Support) version of Node.js.

To install or update Node.js, go to <https://nodejs.org/en/>, download the "LTS" and install it with the default settings.

Close your Terminal (MacOS) or Command Prompt (Windows), re-open it, and run **node-v** again to check the version.

Dependencies

Now that we are in the project folder, we must install dependencies. "What dependencies?" you might ask. We have two dependencies for this project: Vite and Three.js.

To install them, run **npm install** from your terminal.

Wait a little while, and you should see a **node_modules** folder being created in the project folder.

When we installed Node.js, we automatically installed NPM. [NPM](#) is a dependency manager that will fetch the dependencies listed in the **package.json** file and install them in the **node_modules** folder.

Run the server

We are almost ready, and we can now ask Vite to run the server.

To do so, run **npm run dev** while still in Terminal and the project folder.

Wait a second or two, and the website should open in your default browser with "Soon to be a Three.js experience" written on the page.

If the page doesn't open, the terminal should display two URLs, **http://localhost:5173/** and **http://192.168.1.25:5173/**. Try them in your browser.

Terminal folder

Make sure your terminal is opened in the project folder.

Use the **pwd** command to display the terminal's current folder.

Use the **cd** (with a space at the end) command, drag and drop the folder, and press **Enter** to move the terminal into that folder.

Long path

If your project folder is nested deep in other folders, you might have a path so long that Node.js can't handle it.

Move the folder up and move the Terminal accordingly before trying again.

Versioned folder

Be careful with tools like OneDrive, Google Drive, Dropbox, etc., that will “save” your files online.

They can mess up your Node.js dependencies.

Try to move the project outside of those “saved” folders.

Special characters in the path

Avoid having special characters in the path from your computer's root down to the project.

Ideally, you should only have lowercase letters, numbers, dashes, and underscores.

More about the Vite Template

Here are a few things you need to know about the Vite template before we continue:

- The Vite configuration is set in the **vite.config.js** file. If you are curious about configuring a Vite project, check the [Vite website](#).
- You must run the **npm install** only once after downloading the project.
- You need to run **npm run dev** each time you want to run the server and start coding. Your terminal might seem stuck, but it's perfectly normal, meaning the server is running.
- Press **CTRL + C** to stop the server. You might need to press the shortcut multiple times on a Windows system or confirm with the letter **o**.
- The only folders you need to explore, are the **src/** and the **static/** folders.
- In the **src/** folder, you can find the traditional **index.html**, **script.js**, and **style.css** files.
- You can put "static files" in the **static/** folder. Those files will be served as if they were available in the root folder (without having to write **static/**). You can test by adding **/door.jpg** at the end of the URL (**http://localhost:5173/door.jpg**). We'll use this to load textures and models later.
- The page will automatically reload as you save any of those files.
- You can access your local server from any other device on the same network by typing the URL that looks like this **http://192.168.1.25:5173**. It's very useful to debug on other devices like mobiles. If it's not working, it's usually because of your firewall.

Get the Three.js scene back

Here's the easy part. We want to get our Three.js scene back in this Vite template.

First, we need to add the `<canvas>` to the `src/index.html` file:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Local Server</title>
  <link rel="stylesheet" href="./style.css">
</head>
<body>
  <canvas class="webgl"></canvas>
  <script type="module" src="./script.js"></script>
</body>
</html>
```

Now we need to add the same JavaScript code from the previous lesson in the `/src/script.js` file. The only difference is that we need to import Three.js first.

To import Three.js, we need to write the following line at the very start of the `/src/script.js` file:

```
import * as THREE from 'three'
```

This will import all core classes of Three.js inside the **THREE** variable. The **three** modules are in the `/node_modules/` folder, but you don't need to touch them.

Then we can follow with the same code as before:

```
import * as THREE from 'three'

// Scene
const scene = new THREE.Scene()

// Object
const geometry = new THREE.BoxGeometry(1, 1, 1)
const material = new THREE.MeshBasicMaterial({ color: 0xff0000 })
const mesh = new THREE.Mesh(geometry, material)
scene.add(mesh)

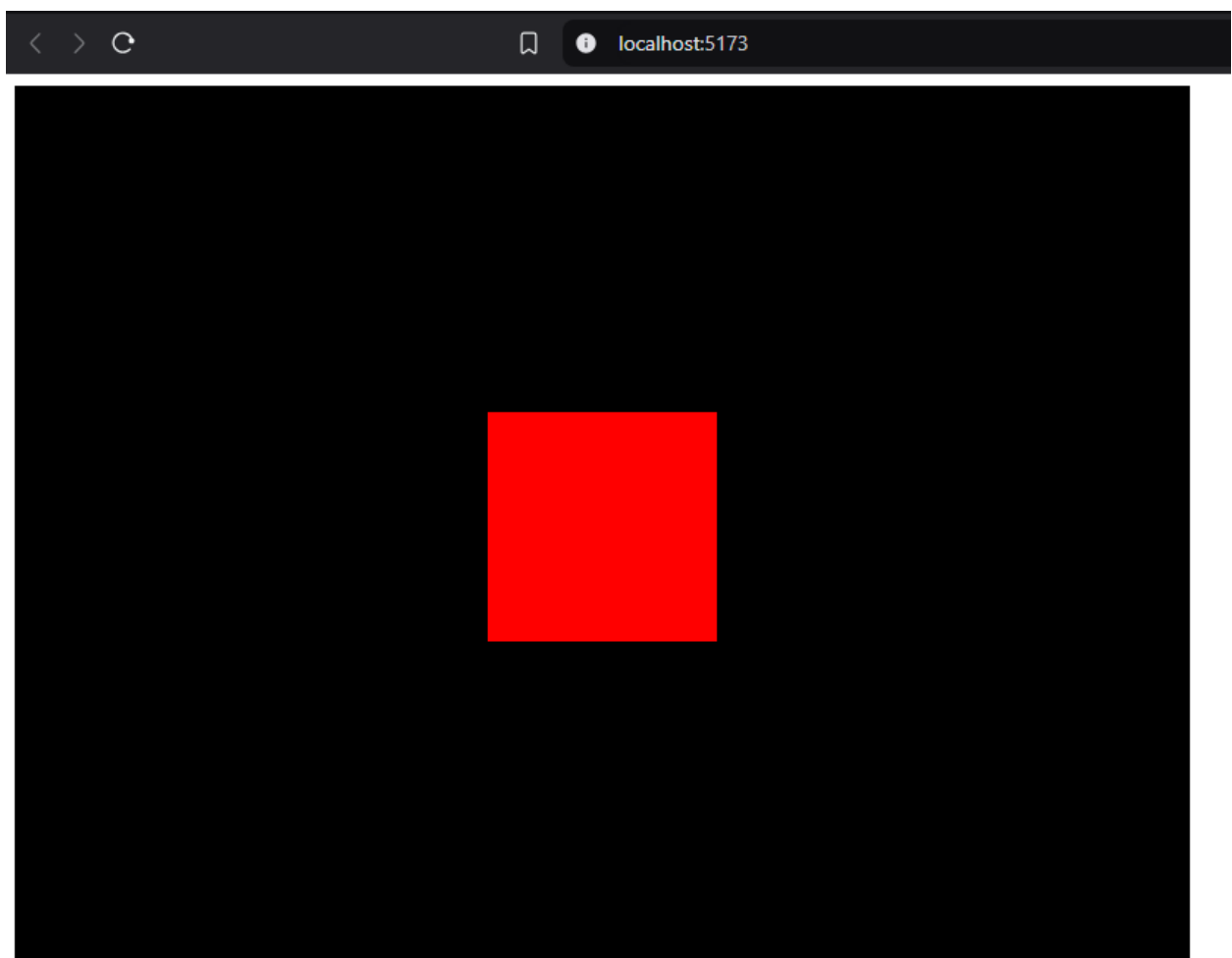
// Sizes
const sizes = {
  width: 800,
  height: 600
}

// Camera
const camera = new THREE.PerspectiveCamera(75, sizes.width / sizes.height)
camera.position.z = 3
scene.add(camera)

// Renderer
const renderer = new THREE.WebGLRenderer({
  canvas: document.querySelector('canvas.webgl')
})
renderer.setSize(sizes.width, sizes.height)
renderer.render(scene, camera)
```

If the server was already running, open the page (no need to reload).

If not, run **npm run dev** from the terminal, and the page should open.



ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 3) TRANSFORM OBJECTS

COURSE/SUBJECT: APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

COURSE NO.: ITE-18

INSTRUCTOR/PROF: JAMES EARL D. CUBILLAS, M.Sc., ECT

Introduction

Now that we have everything in place, we can explore Three.js functionalities.

Before animating our scene, we need to know how to transform objects in our scene. We've already done that with the camera by moving it backward using the **camera.position.z = 3**.

There are 4 properties to transform objects in our scene

- **position** (to move the object)
- **scale** (to resize the object)
- **rotation** (to rotate the object)
- **quaternion** (to also rotate the object; more about that later)

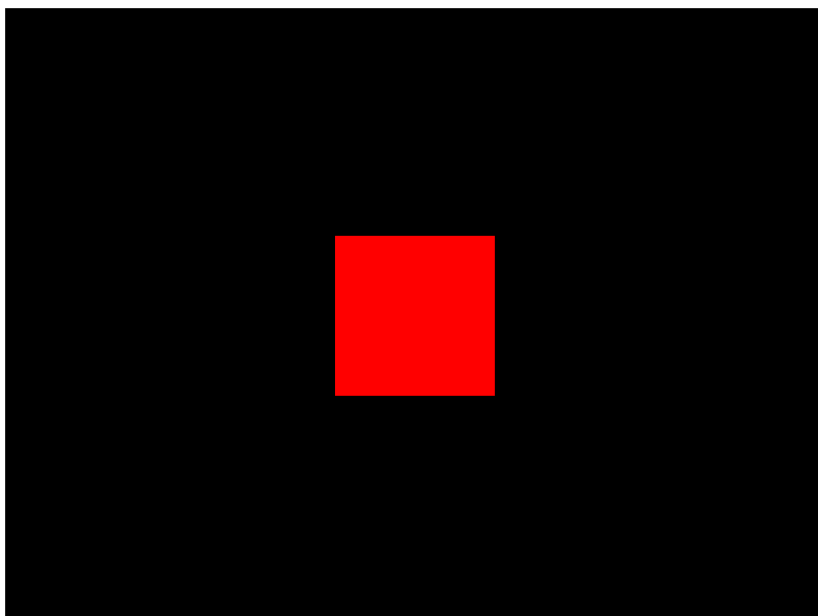
All classes that inherit from the [Object3D](#) class possess those properties like [PerspectiveCamera](#) or [Mesh](#) and classes that we haven't covered yet.

You can see from what classes inherit each class on top of the Three.js documentation.

Those properties will be compiled in what we call matrices. Matrices are used internally by Three.js, by the WebGL, and by the GPU to transform things. Fortunately, you don't have to handle matrices by yourself and you can just modify the previously-mentioned properties.

Setup

In the starter, all we have is the project how we left it in the previous lesson with the cube in the center of the view.



Move objects

The **position** possesses 3 essential properties, which are **x**, **y**, and **z**. Those are the 3 necessary axes to position something in a 3D space.

The direction of each axis is purely arbitrary, and it can vary according to the environment. In Three.js, we usually consider that the **y** axis is going upward, the **z** axis is going backward, and the **x** axis is going to the right.

As for the meaning of **1** unit, it's up to you. **1** can be 1 centimeter, 1 meter, or even 1 kilometer. I recommend that you adapt the unit to what you want to build. If you're going to create a house, you probably should think of **1** unit as 1 meter.

You can play around with the **position** property of your **mesh** and try to guess where the cube will get (try to keep it in the screen).

Make sure to do that before you call the **render(...)** method or you will render the **mesh** before moving it.

```
mesh.position.x = 0.7  
mesh.position.y = - 0.6  
mesh.position.z = 1
```



The **position** property is not any object. It's an instance of the [Vector3](#) class. While this class has an **x**, a **y**, and a **z** property, it also has many useful methods.

You can get the length of a vector:

```
console.log(mesh.position.length())
```

```
1.3601470508735443
```

You can get the distance from another [Vector3](#) (make sure to use this code after creating the camera):

```
console.log(mesh.position.distanceTo(camera.position))
```

```
2.202271554554524
```

You can normalize its values (meaning that you will reduce the length of the vector to **1** unit but preserve its direction):

```
console.log(mesh.position.normalize())
```

```
▼ _Vector3 {x: 0.5146502354656655, y: -0.4411287732562847, z: 0.7352146220938078} ⓘ  
  x: 0.5146502354656655  
  y: -0.4411287732562847  
  z: 0.7352146220938078  
  ► [[Prototype]]: Object
```

To change the values, instead of changing **x**, **y** and **z** separately, you can also use the **set(...)** method:

```
mesh.position.set(0.7, - 0.6, 1)
```

Axes helper

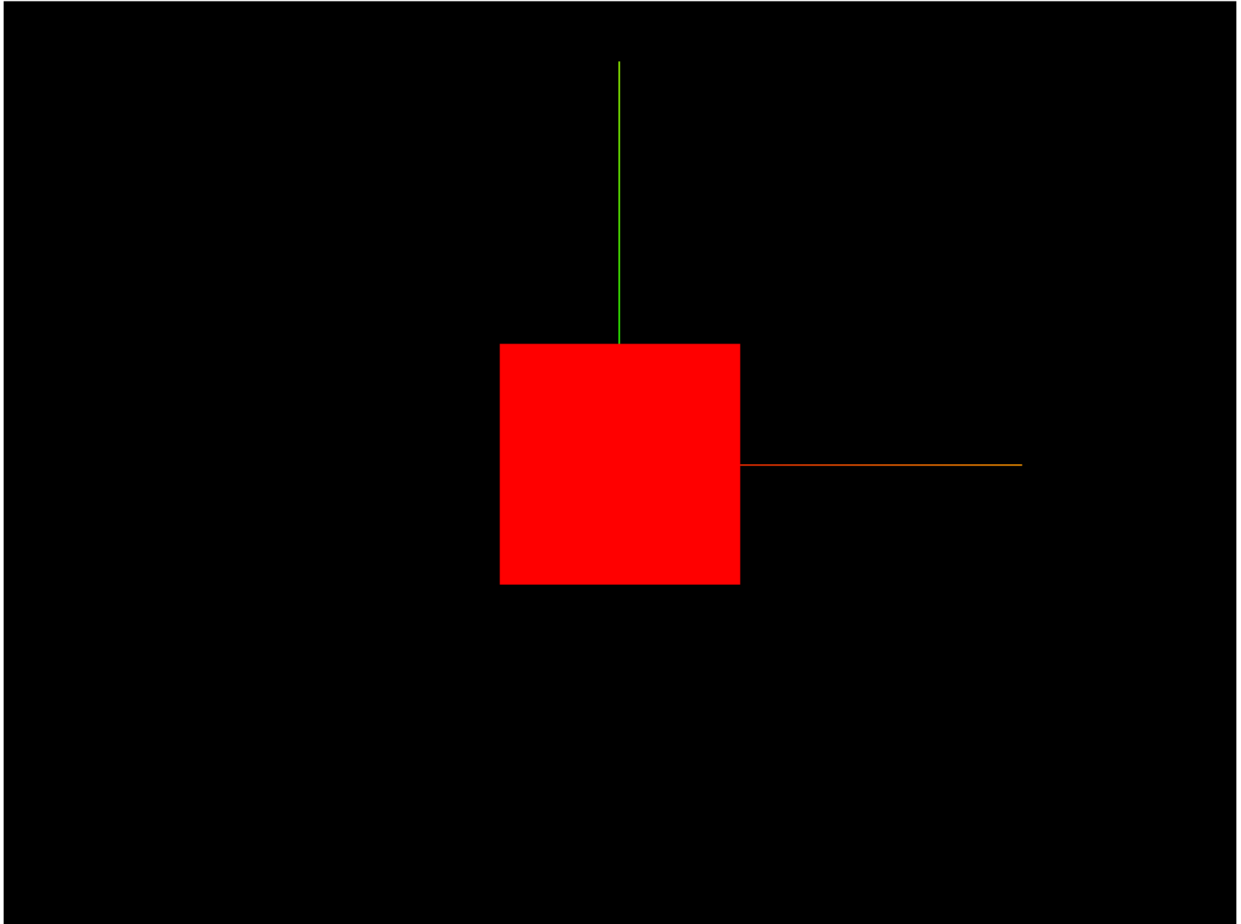
Before we go any further, as you can see, positioning things in space can be a real challenge. Knowing where each axis is oriented is complicated especially when we start to move the camera.

One good solution is to use the Three.js [AxesHelper](#).

The [AxesHelper](#) will display 3 lines corresponding to the **x**, **y** and **z** axes, each one starting at the center of the scene and going in the corresponding direction.

To create the [AxesHelper](#), instantiate it and add it to the **scene** right after instantiating that **scene**. You can specify the length of the lines as the only parameter. We are going to use **2**:

```
/**  
 * Axes Helper  
 */  
const axesHelper = new THREE.AxesHelper(2)  
scene.add(axesHelper)
```



You should see a **green** and a **red** line.

The **green** line corresponds to the **y** axis. The **red** line corresponds to the **x** axis and there is a **blue** line corresponding to the **z** axis but we can't see it because it's perfectly aligned with the camera.

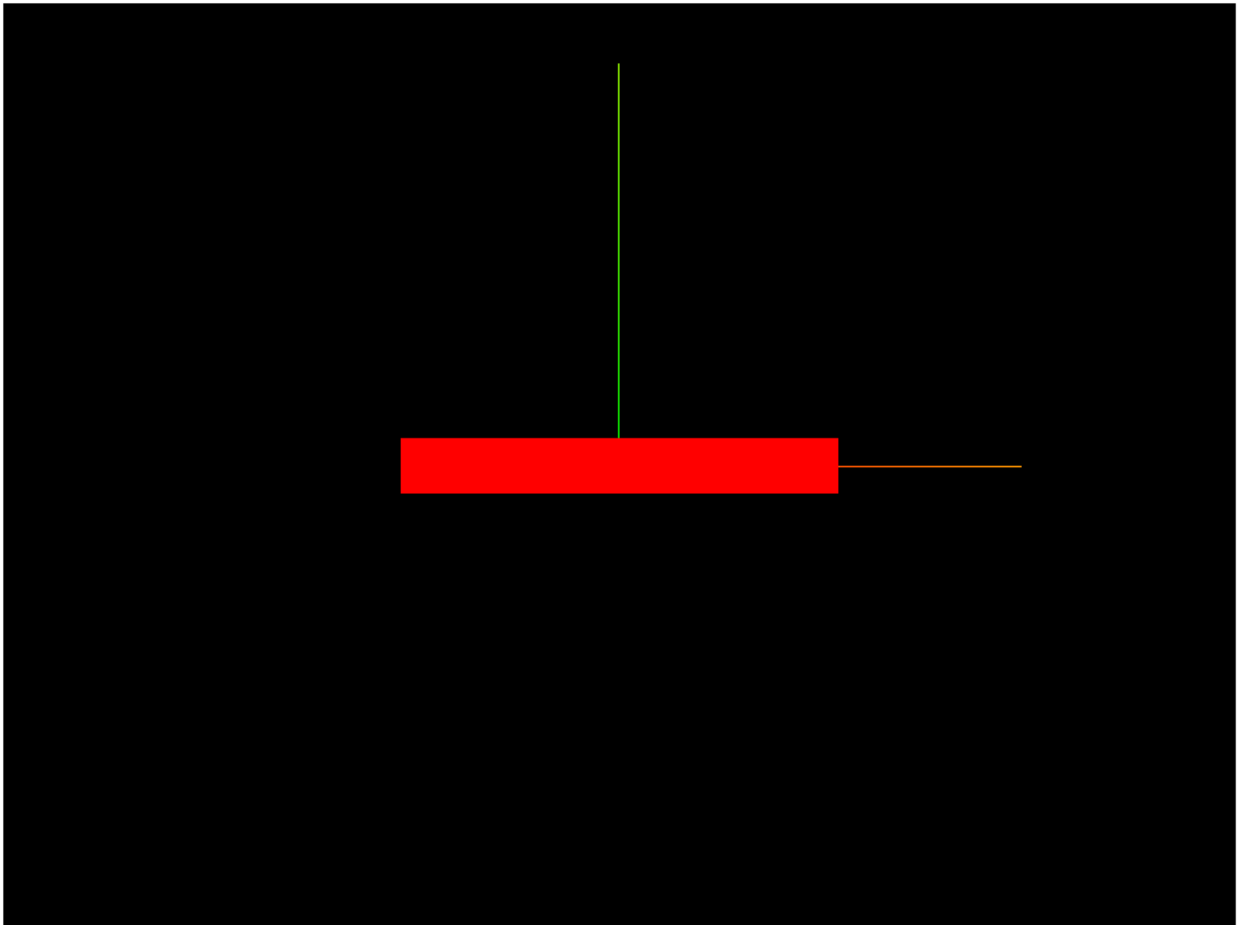
We won't use this [AxesHelper](#) in the next lessons but feel free to add it if you need a visual reference.

Scale objects

scale is also a [Vector3](#). By default, **x**, **y** and **z** are equal to **1**, meaning that the object has no scaling applied. If you put **0.5** as a value, the object will be half of its size on this axis, and if you put **2** as a value, it will be twice its original size on this axis.

If you change those values, the object will start to scale accordingly. Comment the **position** and add these scales:

```
mesh.scale.x = 2
mesh.scale.y = 0.25
mesh.scale.z = 0.5
```



Clearly, we cannot see the **z** scale because our [Mesh](#) is facing the camera.

While you can use negative values, it might generate bugs later on because axes won't be oriented in the logical direction. Try to avoid doing it.

Because it's a [Vector3](#), we can use all the previously mentioned methods.

Rotate objects

Rotation is a little more troublesome than **position** and **scale**. There are two ways of handling a rotation.

You can use the self-evident **rotation** property, but you can also use the less obvious **quaternion** property. Three.js supports both, and updating one will automatically update the other. It's just a matter of which one you prefer.

Rotation

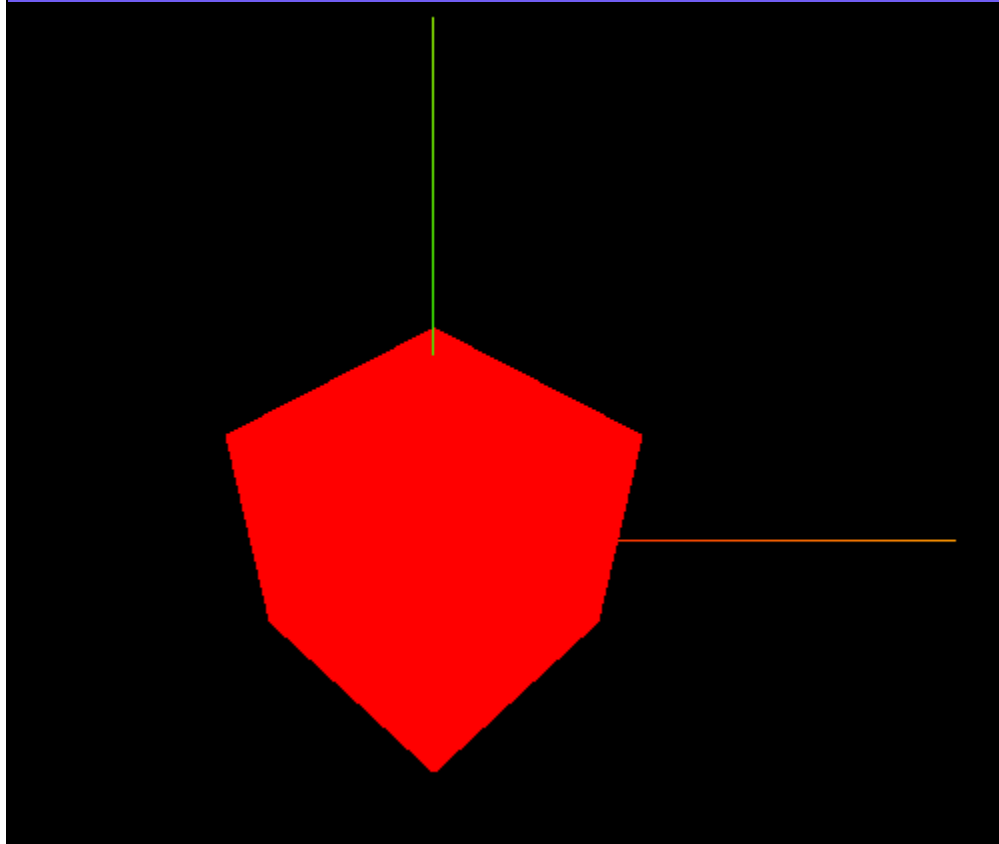
The **rotation** property also has **x**, **y**, and **z** properties, but instead of a [Vector3](#), it's a [Euler](#). When you change the **x**, **y**, and **z** properties of a [Euler](#), you can imagine putting a stick through your object's center in the axis's direction and then rotating that object on that stick.

- If you spin on the **y** axis, you can picture it like a carousel.
- If you spin on the **x** axis, you can imagine that you are rotating the wheels of a car you'd be in.
- And if you rotate on the **z** axis, you can imagine that you are rotating the propellers in front of an aircraft you'd be in.

The value of these axes is expressed in radians. If you want to achieve half a rotation, you'll have to write something like 3.14159... You probably recognize that number as π . In native JavaScript, you can end up with an approximation of π using **Math.PI**.

Comment the **scale** and add an eighth of a complete rotation in both **x** and **y** axes:

```
mesh.rotation.x = Math.PI * 0.25  
mesh.rotation.y = Math.PI * 0.25
```



Is it easy? Yes, but when you combine those rotations, you might end up with strange results. Why? Because, while you rotate the **x** axis, you also change the other axes' orientation. The rotation applies in the following order: **x**, **y**, and then **z**. That can result in weird behaviors like one named gimbal lock when one axis has no more effect, all because of the previous ones.

We can change this order by using the **reorder(...)** method **object.rotation.reorder('YXZ')**

While [Euler](#) is easier to understand, this order problem can cause issues. And this is why most engines and 3D softwares use another solution named [Quaternion](#).

Quaternion

The **quaternion** property also expresses a rotation, but in a more mathematical way, which solves the order problem.

We will not cover how quaternions work in this lesson but keep in mind that the **quaternion** updates when you change the **rotation**. This means that you can use any one of the two as you please.

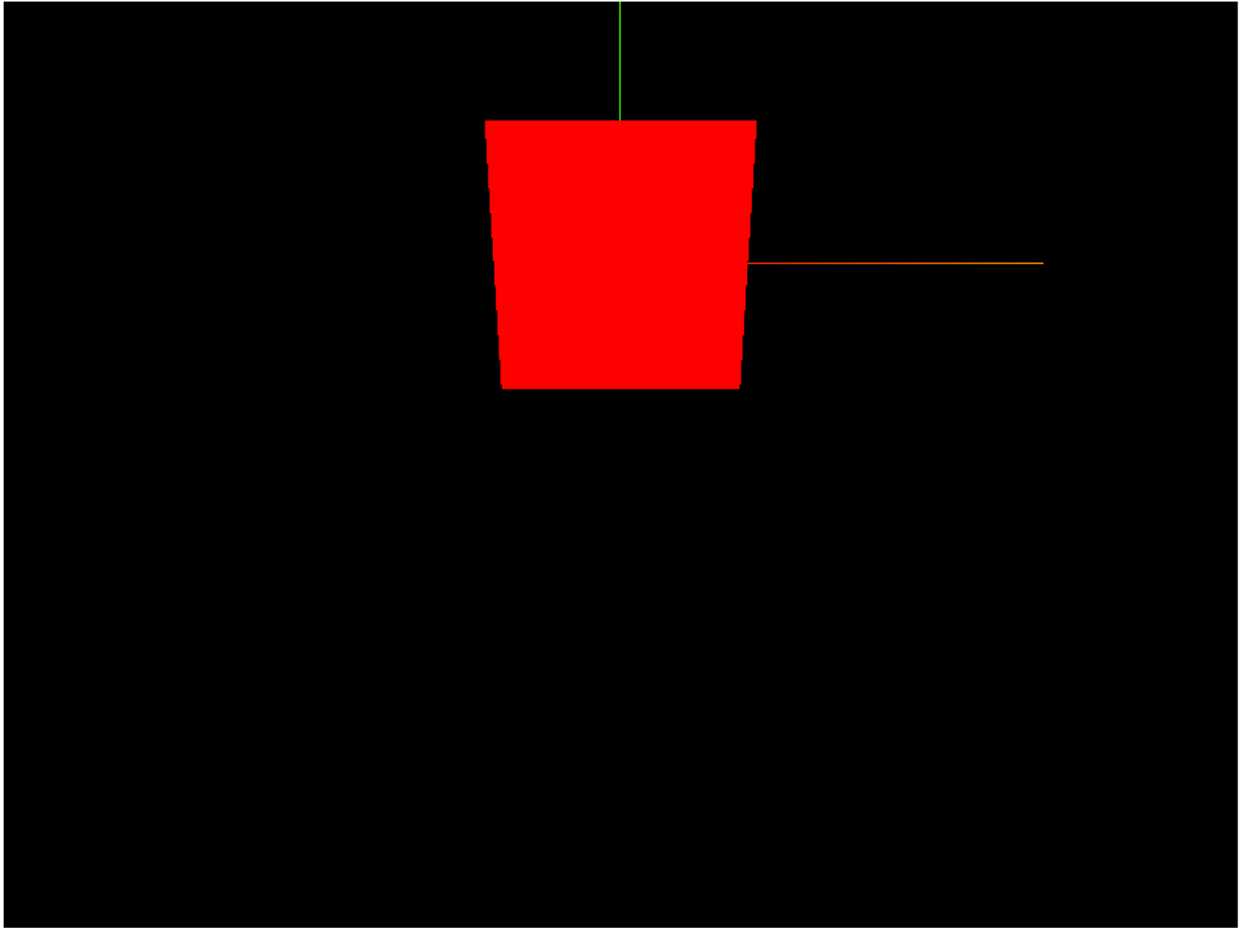
Look at this!

[Object3D](#) instances have an excellent method named **lookAt(...)** that lets you ask an object to look at something. The object will automatically rotate its **-z** axis toward the target you provided. No complicated maths needed.

You can use it to rotate the camera toward an object, orientate a cannon to face an enemy, or move the character's eyes to an object.

The parameter is the target and must be a [Vector3](#). You can create one just to try it:

```
camera.lookAt(new THREE.Vector3(0, - 1, 0))
```



The cube seems to be higher, but in fact, the camera is looking below the cube.

We can also use any existing [Vector3](#) such as the **mesh's position**, but that will result in the default camera position because our **mesh** is in the center of the **scene**.

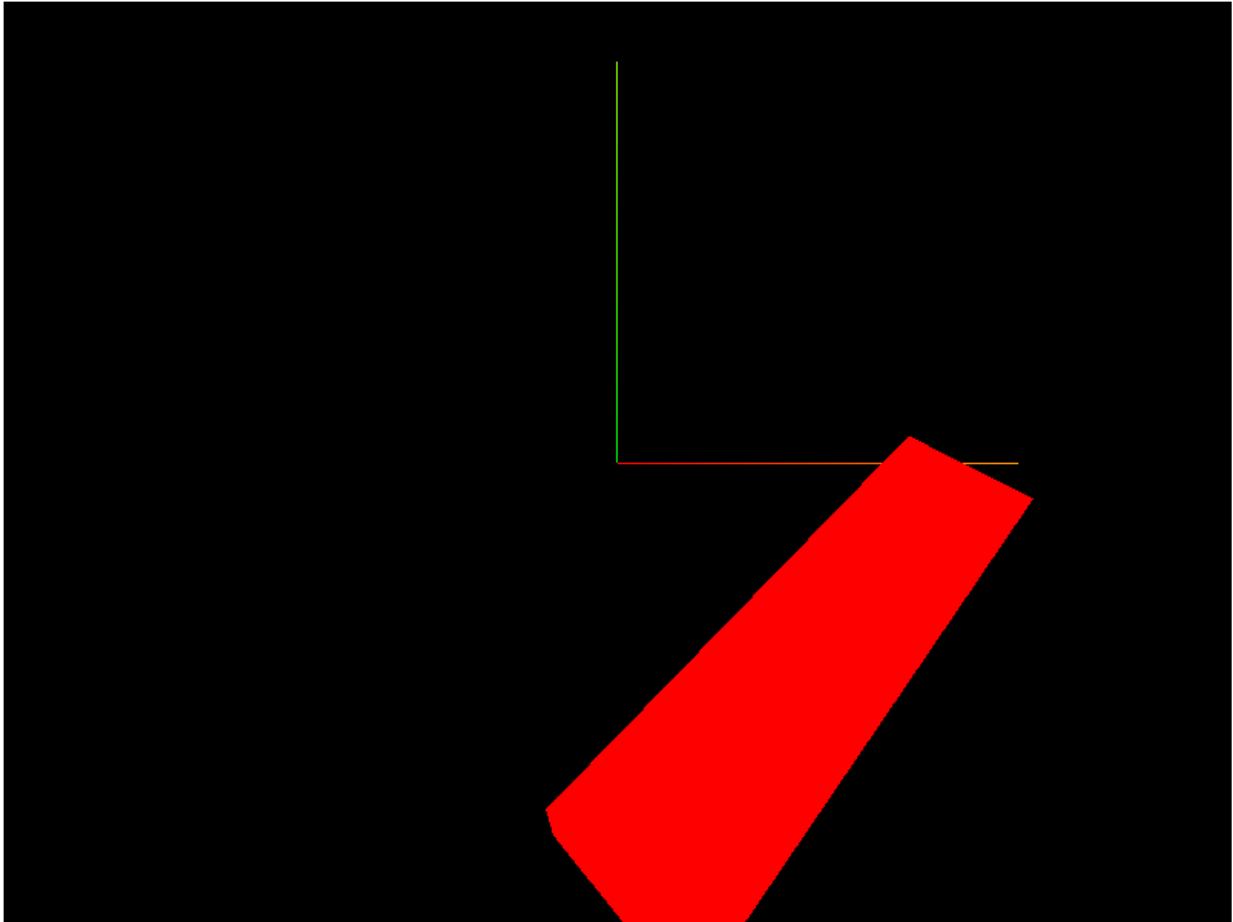
```
camera.lookAt(mesh.position)
```

Combining transformations

You can combine the **position**, the **rotation** (or **quaternion**), and the **scale** in any order. The result will be the same. It's equivalent to the state of the object.

Let's combine all the transformations we tried before:

```
mesh.position.x = 0.7  
mesh.position.y = - 0.6  
mesh.position.z = 1  
mesh.scale.x = 2  
mesh.scale.y = 0.25  
mesh.scale.z = 0.5  
mesh.rotation.x = Math.PI * 0.25  
mesh.rotation.y = Math.PI * 0.25
```



Scene graph

At some point, you might want to group things. Let's say you are building a house with walls, doors, windows, a roof, bushes, etc.

When you think you're done, you become aware that the house is too small, and you have to re-scale each object and update their positions.

A good alternative would be to group all those objects into a container and scale that container.

You can do that with the [Group](#) class.

Instantiate a [Group](#) and add it to the scene. Now, when you want to create a new object, you can add it to the [Group](#) you just created using the **add(...)** method rather than adding it directly to the scene

Because the [Group](#) class inherits from the [Object3D](#) class, it has access to the previously-mentioned properties and methods like **position**, **scale**, **rotation**, **quaternion**, and **lookAt**.

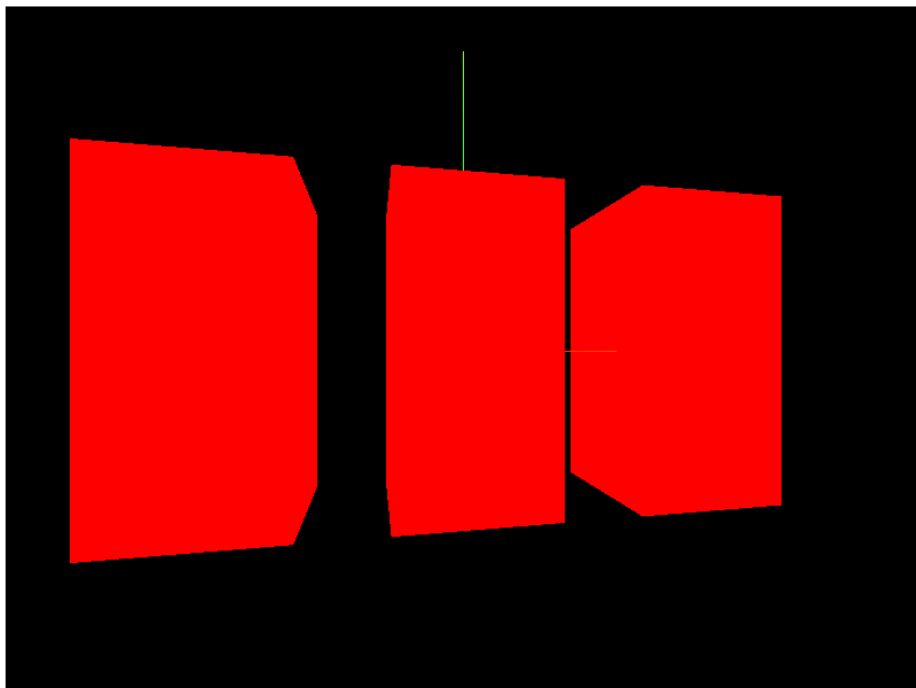
Comment the **lookAt(...)** call and, instead of our previously created cube, create 3 cubes and add them to a [Group](#). Then apply transformations on the **group**:

```
/**
 * Objects
 */
const group = new THREE.Group()
group.scale.y = 2
group.rotation.y = 0.2
scene.add(group)

const cube1 = new THREE.Mesh(
  new THREE.BoxGeometry(1, 1, 1),
  new THREE.MeshBasicMaterial({ color: 0xff0000 })
)
cube1.position.x = - 1.5
group.add(cube1)

const cube2 = new THREE.Mesh(
  new THREE.BoxGeometry(1, 1, 1),
  new THREE.MeshBasicMaterial({ color: 0xff0000 })
)
cube2.position.x = 0
group.add(cube2)

const cube3 = new THREE.Mesh(
  new THREE.BoxGeometry(1, 1, 1),
  new THREE.MeshBasicMaterial({ color: 0xff0000 })
)
cube3.position.x = 1.5
group.add(cube3)
```



The order doesn't really matter, as long as it's valid JavaScript.

Now that we know how to transform objects, it's time to create some animations.

ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 4) ANIMATIONS

COURSE/SUBJECT: APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

COURSE NO.: ITE-18

INSTRUCTOR/PROF: JAMES EARL D. CUBILLAS, M.Sc., ECT

Introduction

We created a scene that we rendered once at the end of our code. That is already good progress, but you'll want to animate your creations most of the time.

Animations, when using Three.js, work like stop motion. You move the objects, and you do a render. Then you move the objects a little more, and you do another render. Etc. The more you move the objects between renders, the faster they'll appear to move.

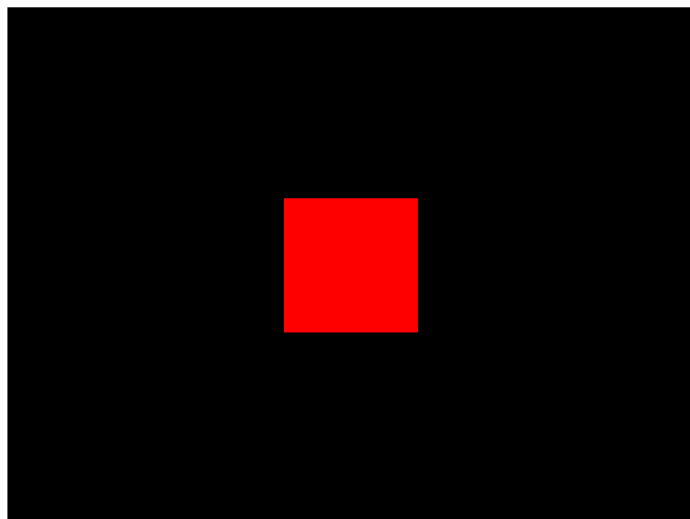
The screen you are looking at runs at a specific frequency. We call that a frame rate. The frame rate mostly depends on the screen, but the computer has limitations. Most screens run at 60 frames per second. If you do the maths, that means about a frame every 16ms. But some screens can run much faster, and when the computer has difficulties processing things, it'll run more slowly.

We want to execute a function that will move objects and do the render on each frame regardless of the frame rate.

The native JavaScript way of doing so is by using the **`window.requestAnimationFrame(...)`** method.

Setup

As we had before, all we have in the starter is our cube in the scene's center.



Using requestAnimationFrame

The primary purpose of [requestAnimationFrame](#) is not to run code on each frame.

`requestAnimationFrame` will execute the function you provide **on the next frame**. But then, if this function also uses **`requestAnimationFrame`** to execute that same function on the next frame, you'll end up with your function being executed on each frame forever which is exactly what we want.

Create a function named **tick** and call this function once. In this function, use **window.requestAnimationFrame(...)** to call this same function on the next frame:

```
/**
 * Animate
 */
const tick = () =>
{
  console.log('tick')

  window.requestAnimationFrame(tick)
}

tick()
```

That's it. You have your infinite loop.

As you can see on the console, the '**tick**' is being called on each frame. If you test this code on a computer with a high frame rate, the '**tick**' will appear at a higher frequency.

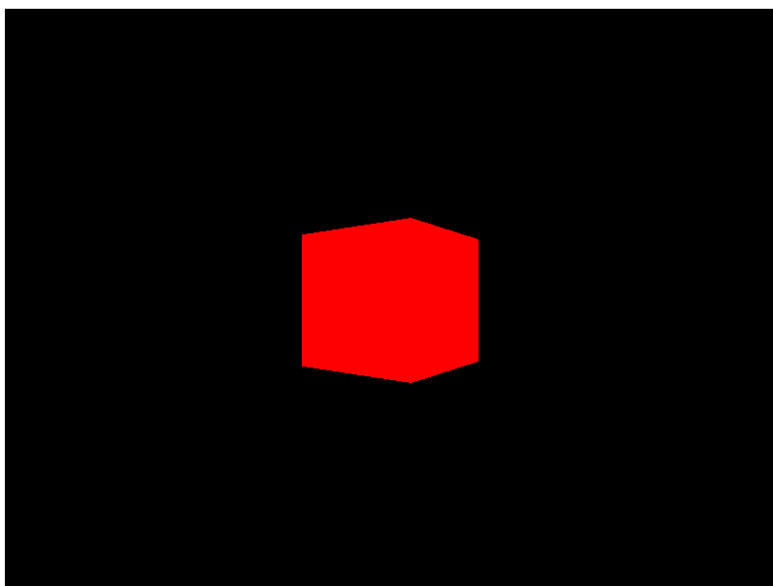
You can now move the **renderer.render(...)** call inside that function and increase the cube **rotation**:

```
/**
 * Animate
 */
const tick = () =>
{
  // Update objects
  mesh.rotation.y += 0.01

  // Render
  renderer.render(scene, camera)

  // Call tick again on the next frame
  window.requestAnimationFrame(tick)
}

tick()
```



Congratulations, you now have a Three.js animation.

The problem is, if you test this code on a computer with high frame rate, the cube will rotate faster, and if you test on a lower frame rate, the cube will rotate slower.

Adaptation to the framerate

To adapt the animation to the framerate, we need to know how much time it's been since the last tick.

First, we need a way to measure time. In native JavaScript, you can use **Date.now()** to get the current timestamp:

```
const time = Date.now()
```

The timestamp corresponds to how much time has passed since the 1st of January 1970 (the beginning of time for Unix). In JavaScript, its unit is in milliseconds.

What you need now is to subtract the current timestamp to that of the previous frame to get what we can call the **deltaTime** and use this value when animating objects:

```
/**
 * Animate
 */
let time = Date.now()

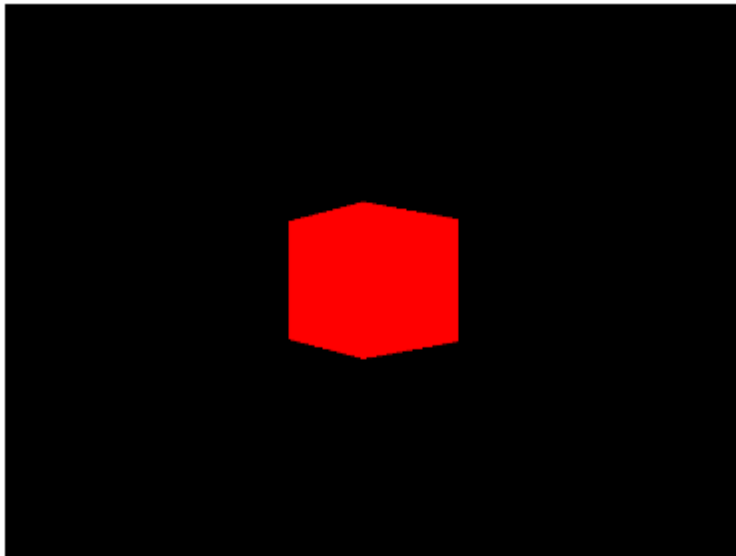
const tick = () =>
{
    // Time
    const currentTime = Date.now()
    const deltaTime = currentTime - time
    time = currentTime

    // Update objects
    mesh.rotation.y += 0.01 * deltaTime

    // Render
    renderer.render(scene, camera)

    // Call tick again on the next frame
    window.requestAnimationFrame(tick)
}

tick()
```



The cube should rotate faster because the `deltaTime` should be around **16** if your screen is running at **60fps**, so feel free to reduce it by multiplying the value.

Now that we base our rotation on how much time was spent since the last frame, this rotation speed will be the same on every screen and every computers regardless of the frame rate.

Using Clock

While this code isn't that complicated, there is a built-in solution in Three.js named [Clock](#) that will handle the time calculations.

You simply have to instantiate a [Clock](#) variable and use the built-in methods like `getElapsedTime()`. This method will return how many seconds have passed since the [Clock](#) was created.

You can use this value to rotate the object:

```
/**
 * Animate
 */
const clock = new THREE.Clock()

const tick = () =>
{
  const elapsedTime = clock.getElapsedTime()

  // Update objects
  mesh.rotation.y = elapsedTime

  // Render
  renderer.render(scene, camera)

  // Call tick again on the next frame
  window.requestAnimationFrame(tick)
}

tick()
```

You can also use it to move things with the **position** property. If you combine it with **Math.sin(...)** you can get a pretty good result:

```
/**
 * Animate
 */
const clock = new THREE.Clock()

const tick = () =>
{
    const elapsedTime = clock.getElapsedTime()

    // Update objects
    mesh.position.x = Math.cos(elapsedTime)
    mesh.position.y = Math.sin(elapsedTime)

    // Render
    renderer.render(scene, camera)

    // Call tick again on the next frame
    window.requestAnimationFrame(tick)
}

tick()
```

And obviously, you can use those techniques to animate any [Object3D](#) like the camera:

```
/**
 * Animate
 */
const clock = new THREE.Clock()

const tick = () =>
{
    const elapsedTime = clock.getElapsedTime()

    // Update objects
    camera.position.x = Math.cos(elapsedTime)
    camera.position.y = Math.sin(elapsedTime)
    camera.lookAt(mesh.position)

    // Render
    renderer.render(scene, camera)

    // Call tick again on the next frame
    window.requestAnimationFrame(tick)
}

tick()
```

Another available method is `getDelta(...)`, but you should not use it unless you know exactly what's going on in the [Clock](#) class code. Using it might mess with your animation, and you'll get unwanted results.

Using a library

Sometimes you'll want to animate your scene in a very specific way that will require using another library. There are tons of animation libraries, but a very famous one is [GSAP](#).

To add GSAP to our project, we can use the dependency manager provided with Node.js called **npm**.

In your terminal (while the server is not running or by using another terminal window on the same folder), run **npm install --save gsap@3.5.1**

The **--save** argument saves the dependency in the **package.json** so the module can be fetched if we do an **npm install**.

The **@3.5.1** forces the version. We use this version because it was the one used when writing the lesson, but you can try the latest version if you want by removing **@3.5.1**.

GSAP is now available in the **node_modules/** folder, and we can import it in our **script.js**:

```
import './style.css'
import * as THREE from 'three'
import gsap from 'gsap'

// ...
```

There are many ways of using GSAP, and we could dedicate an entire course to it, but it is not the goal of this course. We will simply create a tween to test things out. If you already know how to use GSAP, it works the same with Three.js.

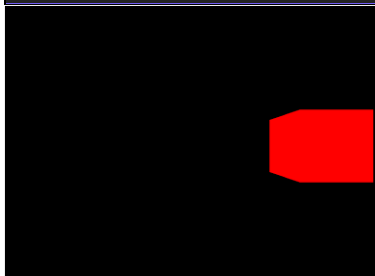
Comment the code related to the previous animations but keep the **tick** function with the render. Then you can create what we call a tween (an animation from A to B) using **gsap.to(...)**:

```
/**
 * Animate
 */
gsap.to(mesh.position, { duration: 1, delay: 1, x: 2 })

const tick = () =>
{
  // Render
  renderer.render(scene, camera)

  // Call tick again on the next frame
  window.requestAnimationFrame(tick)
}

tick()
```



GSAP has a built-in **requestAnimationFrame**, so you don't need to update the animation by yourself, but still, if you want to see the cube moving, you need to keep doing the renders of your scene on each frame.

Choosing the right solution

As for choosing between native JS and an animation library, it's a matter of what you want to achieve. If you're going to create a carousel that spins forever, you don't need any library for that. But if you want to animate, for instance, the swing of a sword, you might prefer to use a library.

ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 5) CAMERAS

COURSE/SUBJECT: APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES

COURSE NO.: ITE-18

INSTRUCTOR/PROF: JAMES EARL D. CUBILLAS, M.Sc., ECT

Introduction

We already created a **PerspectiveCamera**, but there are other types of cameras, as you can see in the documentation.

Camera

The [Camera](#) class is what we call an abstract class. You're not supposed to use it directly, but you can inherit from it to have access to common properties and methods. Some of the following classes inherit from the [Camera](#) class.

ArrayCamera

The [ArrayCamera](#) is used to render your scene multiple times by using multiple cameras. Each camera will render a specific area of the canvas. You can imagine this looking like old school console multiplayer games where we had to share a split-screen.

StereoCamera

The [StereoCamera](#) is used to render the scene through two cameras that mimic the eyes in order to create what we call a parallax effect that will lure your brain into thinking that there is depth. You must have the adequate equipment like a VR headset or red and blue glasses to see the result.

CubeCamera

The [CubeCamera](#) is used to get a render facing each direction (forward, backward, leftward, rightward, upward, and downward) to create a render of the surrounding. You can use it to create an environment map for reflection or a shadow map. We'll talk about those later.

OrthographicCamera

The [OrthographicCamera](#) is used to create orthographic renders of your scene without perspective. It's useful if you make an RTS game like Age of Empire. Elements will have the same size on the screen regardless of their distance from the camera.

PerspectiveCamera

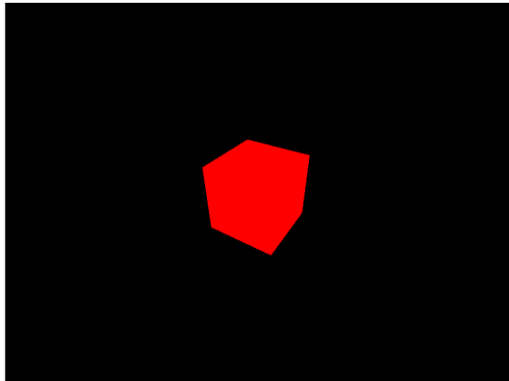
The [PerspectiveCamera](#) is the one we already used and simulated a real-life camera with perspective.

We are going to focus on the [OrthographicCamera](#) and the [PerspectiveCamera](#).

PerspectiveCamera

As we saw earlier, the [PerspectiveCamera](#) class needs some parameters to be instantiated, but we didn't use all the possible parameters. Add the third and fourth parameters:

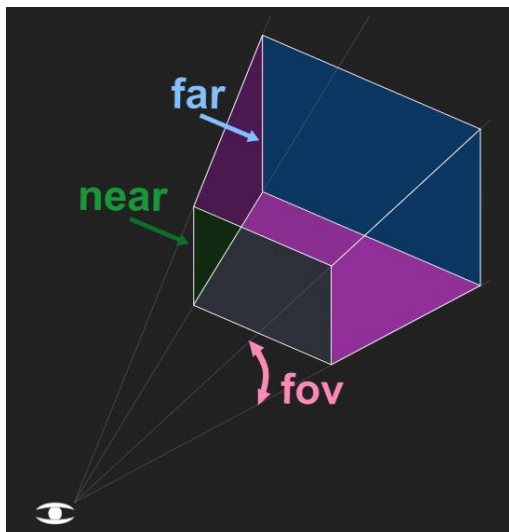
```
const camera = new THREE.PerspectiveCamera(75, sizes.width / sizes.height, 1, 100)
```



You should get the same result but let's talk about those parameters in detail.

Field of view

The first parameter called **field of view** corresponds to your camera view's vertical amplitude angle in degrees. If you use a small angle, you'll end up with a long scope effect, and if you use a wide-angle, you'll end up with a fish eye effect because, in the end, what the camera sees will be stretched or squeezed to fit the canvas.



As for choosing the right field of view, you'll have to try things out. I usually use a field of view between **45** and **75**.

Aspect ratio

The second parameter is called **aspect ratio** and corresponds to the width divided by the height. While you might think that it's obviously the canvas width by the canvas height and Three.js should calculate it by itself, it's not always the case if you start using Three.js in very specific ways. But in our case, you can simply use the canvas width and the canvas height.

I recommend saving those values in an object because we are going to need them multiple times:

```
const sizes = {
  width: 800,
  height: 600
}
```

Near and far

The third and fourth parameters called **near** and **far**, correspond to how close and how far the camera can see. Any object or part of the object closer to the camera than the **near** value or further away from the camera than the **far** value will not show up on the render.

You can see that like in those old racing games where you could see the trees pop up in the distance.

While you might be tempted to use very small and very large values like **0.0001** and **9999999** you might end up with a bug called z-fighting where two faces seem to fight for which one will be rendered above the other.

<https://twitter.com/FreyaHolmer/status/799602767081848832>

https://twitter.com/Snapman_I_Am/status/800567120765616128

Try to use reasonable values and increase those only if you need it. In our case, we can use **0.1** and **100**.

OrthographicCamera

While we will not use this type of camera for the rest of the course, it can be useful for specific projects.

The [OrthographicCamera](#) differs from the [PerspectiveCamera](#) by its lack of perspective, meaning that the objects will have the same size regardless of their distance from the camera.

The parameters you have to provide are very different from the [PerspectiveCamera](#).

Instead of a field of view, you must provide how far the camera can see in each direction (**left**, **right**, **top** and **bottom**). Then you can provide the **near** and **far** values just like we did for the [PerspectiveCamera](#).

Comment the [PerspectiveCamera](#) and add [OrthographicCamera](#) . Keep the **position** update and **lookAt(...)** call:

```
const camera = new THREE.OrthographicCamera(- 1, 1, 1, - 1, 0.1, 100)
```

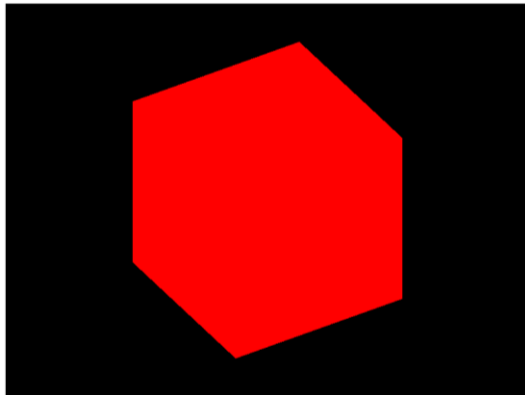


As you can see, there is no perspective, and the sides of our cube seem parallel. The problem is that our cube doesn't look cubic.

That is due to the values we provided for the **left**, **right**, **top**, and **bottom** , which are 1 or - 1, meaning that we render a square area, but that square area will be stretched to fit our rectangle canvas, and our canvas isn't a square.

We need to use the canvas ratio (width by height). Let's create a variable named **aspectRatio** (just like the `PerspectiveCamera`) and store that ratio in it:

```
const aspectRatio = sizes.width / sizes.height
const camera = new THREE.OrthographicCamera(- 1 * aspectRatio, 1 *
aspectRatio, 1, - 1, 0.1, 100)
```



This results in a render area width larger than the render area height because our canvas width is larger than its height.

We now have a cube that looks like a cube.

Custom controls

Let's get back to our [PerspectiveCamera](#). Comment the [OrthographicCamera](#), uncomment the [PerspectiveCamera](#), move the **camera** so it faces the cube, and remove the mesh rotation in the **tick** function:

```
// Camera
const camera = new THREE.PerspectiveCamera(75, sizes.width / sizes.height,
1, 1000)

// const aspectRatio = sizes.width / sizes.height
// const camera = new THREE.OrthographicCamera(- 1 * aspectRatio, 1 *
aspectRatio, 1, - 1, 0.1, 100)

// camera.position.x = 2
// camera.position.y = 2
camera.position.z = 3
camera.lookAt(mesh.position)
scene.add(camera)
```

What we want to do now is control the camera with our mouse. First of all, we want to know the mouse coordinates. We can do that using native JavaScript by listening to the **mousemove** event with **addEventListener**.

The coordinates will be located in the argument of the callback function as **event.clientX** and **event.clientY**:

```
// Cursor
window.addEventListener('mousemove', (event) =>
{
    console.log(event.clientX, event.clientY)
})
```

We could use those values, but I recommend adjusting them. By adjusting, I mean to have a **1** amplitude and that the value can be both negative and positive.

If we only focus on the **x** value, that would mean that:

- if your cursor is on the far left of the canvas, you should get **- 0.5**
- if your cursor is at the center of the canvas, you should get **0**
- if your cursor is at the far right of the canvas, you should get **0.5**

While this is not mandatory, it helps to have clean values like that.

Just like the **size** variable, we will create a **cursor** variable with default **x** and **y** properties and then update those properties in the **mousemove** callback:

```
// Cursor
const cursor = {
    x: 0,
    y: 0
}

window.addEventListener('mousemove', (event) =>
{
    cursor.x = event.clientX / sizes.width - 0.5
    cursor.y = event.clientY / sizes.height - 0.5

    console.log(cursor.x, cursor.y)
})
```

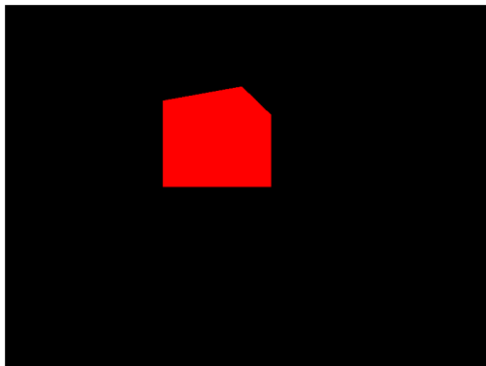
Dividing **event.clientX** by **sizes.width** will give us a value between **0** and **1** (if we keep the cursor above the canvas) while subtracting **0.5** will give you a value between **- 0.5** and **0.5**.

You now have the mouse position stored in the **cursor** object variable, and you can update the position of the camera in the **tick** function:

```
const tick = () =>
{
    // ...

    // Update camera
    camera.position.x = cursor.x
    camera.position.y = cursor.y

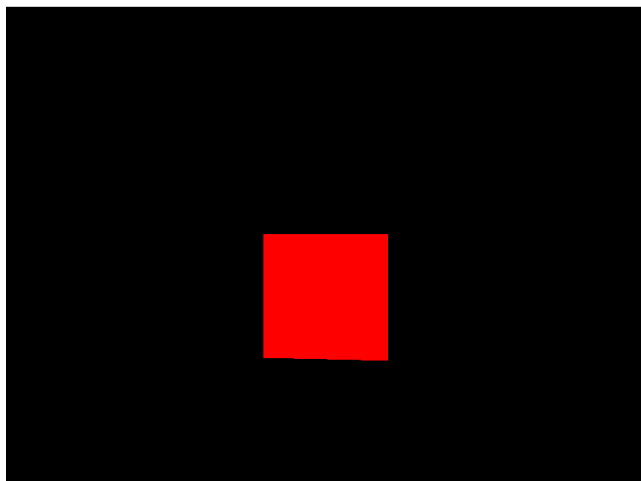
    // ...
}
```



As you can see, it's working but the axes movements seem kind of wrong. This is due to the **position.y** axis being positive when going upward in Three.js but the **clientY** axis being positive when going downward in the webpage.

You can simply invert the **cursor.y** while updating it by adding a - in front of the whole formula (don't forget the parentheses):

```
window.addEventListener('mousemove', (event) =>
{
  cursor.x = event.clientX / sizes.width - 0.5
  cursor.y = - (event.clientY / sizes.height - 0.5)
})
```

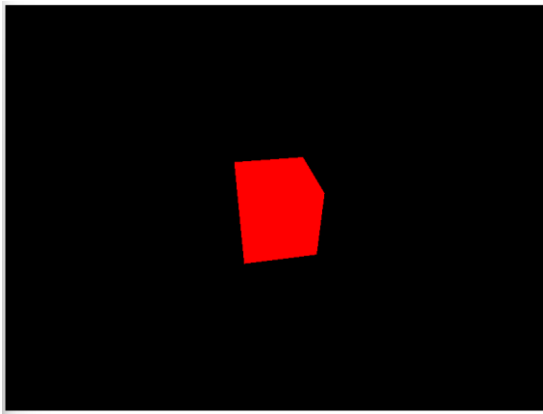


Finally, you can increase the amplitude by multiplying the **cursor.x** and **cursor.y** and ask the camera to look at the mesh using the **lookAt(...)** method:

```
const tick = () =>
{
  // ...

  // Update camera
  camera.position.x = cursor.x * 5
  camera.position.y = cursor.y * 5
  camera.lookAt(mesh.position)

  // ...
}
```



We can go even further by doing a full rotation of the camera around the mesh by using **Math.sin(...)** and **Math.cos(...)**.

sin and **cos**, when combined and used with the same angle, enable us to place things on a circle. To do a full rotation, that angle must have an amplitude of 2 times π (called "pi"). Just so you know, a full rotation is called a "tau" but we don't have access to this value in JavaScript and we have to use π instead.

You can access an approximation of π in native JavaScript using **Math.PI**.

To increase the radius of that circle, you can simply multiply the result of **Math.sin(...)** and **Math.cos(...)**:

```
const tick = () =>
{
  // ...

  // Update camera
  camera.position.x = Math.sin(cursor.x * Math.PI * 2) * 2
  camera.position.z = Math.cos(cursor.x * Math.PI * 2) * 2
  camera.position.y = cursor.y * 3
  camera.lookAt(mesh.position)

  // ...
}

tick()
```

While this is a good start to control the camera, Three.js has integrated multiple classes called controls to help you do the same and much more.

Built-in controls

If you type "controls" in the [Three.js documentation](#), you'll see that there are a lot of pre-made controls. We will only use one of them for the rest of the course, but it can be interesting to know their role.

DeviceOrientationControls

[DeviceOrientationControls](#) will automatically retrieve the device orientation if your device, OS, and browser allow it and rotate the camera accordingly. You can use it to create immersive universes or VR experiences if you have the right equipment.

FlyControls

[FlyControls](#) enable moving the camera like if you were on a spaceship. You can rotate on all 3 axes, go forward and go backward.

FirstPersonControls

[FirstPersonControls](#) is just like [FlyControls](#), but with a fixed up axis. You can see that like a flying bird view where the bird cannot do a barrel roll. While the FirstPersonControls contains "FirstPerson," it doesn't work like in FPS games.

PointerLockControls

[PointerLockControls](#) uses the [pointer lock JavaScript API](#). This API hides the cursor, keeps it centered, and keeps sending the movements in the `mousemove` event callback. With this API, you can create FPS games right inside the browser. While this class sounds very promising if you want to create that kind of interaction, it'll only handle the camera rotation when the pointer is locked. You'll have to handle the camera position and game physics by yourself.

OrbitControls

[OrbitControls](#) is very similar to the controls we made in the previous lesson. You can rotate around a point with the left mouse, translate laterally using the right mouse, and zoom in or out using the wheel.

TrackballControls

[TrackballControls](#) is just like [OrbitControls](#) but there are no limits in terms of vertical angle. You can keep rotating and do spins with the camera even if the scene gets upside down.

TransformControls

[TransformControls](#) has nothing to do with the camera. You can use it to add a gizmo to an object to move that object.

DragControls

Just like the [TransformControls](#), [DragControls](#) has nothing to do with the camera. You can use it to move objects on a plane facing the camera by drag and dropping them.

We will only use the [OrbitControls](#) but feel free to test the other classes.

OrbitControls

Let's comment the part where we update the **camera** in the **tick** function.

Instantiating

First, we need to instantiate a variable using the [OrbitControls](#) class. While you might think you can use **THREE.OrbitControls** here, you are unfortunately mistaken.

The OrbitControls class is part of those classes that are not available by default in the **THREE** variable. That decision helps to reduce the weight of the library. And this is where our Vite template comes in.

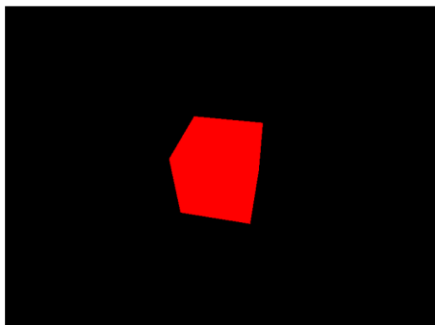
The **OrbitControls** class may not be available in the **THREE** variable; it is still located in the dependencies folder. To import it, you must provide the path from inside the **/node_modules/** folder, which is **/three/examples/jsm/controls/OrbitControls.js**:

```
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
```

You can now instantiate a variable using the class **OrbitControls** (without the **THREE**.) and make sure to do that after creating the camera.

For it to work, you must provide the camera and the element in the page that will handle the mouse events as parameters:

```
// Controls
const controls = new OrbitControls(camera, canvas)
```



You can now drag and drop using both the left mouse or the right mouse to move the camera, and you can scroll up or down to zoom in or out.

It's much easier than our custom code, and it comes with more controls. But let's go a little further.

Target

By default, the camera is looking at the center of the scene. We can change that with the **target** property.

This property is a [Vector3](#), meaning that we can change its **x**, **y** and **z** properties.

If we want the [OrbitControls](#) to look above the cube by default, we just have to increase the **y** property:

```
controls.target.y = 2
```

But this won't work just like that because we need to tell the **OrbitControl** to update itself. And we can do that by calling the **update** method right after:

```
controls.target.y = 2  
controls.update()
```

This is not very useful in our case so let's comment this part.

Damping

If you read the documentation of [OrbitControls](#) there are mentions of **damping**. The damping will smooth the animation by adding some kind of acceleration and friction formulas.

To enable damping, switch the **enableDamping** property of **controls** to **true**.

In order to work properly, the controls also needs to be updated on each frame by calling **controls.update()**. You can do that on the **tick** function:

```
// Controls  
const controls = new OrbitControls(camera, canvas)  
controls.enableDamping = true  
  
// ...  
  
const tick = () =>  
{  
  // ...  
  
  // Update controls  
  controls.update()  
  
  // ...  
}
```

You'll see that the controls are now a lot smoother.

You can use many other methods and properties to customize your controls such as the rotation speed, zoom speed, zoom limit, angle limit, damping strength, and key bindings (because yes, you can also use your keyboard).

When to use built-in controls

While those controls are handy, they have limitations. If you rely too much on them, you might end up having to change how the class is working in an unexpected way.

First, make sure to list all the features you need from those controls, then check if the class you're about to use can handle all of those features.

If not, you'll have to do it on your own.